VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY



FACULTY OF INFORMATION TECHNOLOGY

LAB REPORT

Lab 01: Xv6 User Programs

Instructor:
Leader: Phuc-Khang Nguyen 24120068
Members: Minh-Trong Pham 24120151
Trong-Nghia Hoang 24120103

Ho Chi Minh City, Ngày 16 tháng 10 năm 2026

Table of Contents

1 primes.c **1**

1.1 Tổng quan 1

1.2 Khó khăn và thử thách 2

2 tree.c **3**

2.1 Tổng quan 3

2.2 Khó khăn và thử thách 4

3 diff.c **7**

3.1 Tổng quan 7

3.2 Khó khăn và Thử thách 8

4 du.c **9**

4.1 Tổng quan 9

4.2 Khó khăn và Thử thách 11

5 pingpong.c **13**

5.1 Tổng quan 13

5.2 Khó khăn và Thử thách 14

6 cp.c **15**

6.1 Tổng quan 15

6.2 Khó khăn và Thử thách 17

1 primes.c

1.1 Tổng quan

`primes.c` hiện thực thuật toán Sàng Eratosthenes đồng thời (*Concurrent Prime Sieve*) để tìm các số nguyên tố trong khoảng 2 đến 280. Thay vì lọc tuần tự trong một tiến trình, chương trình tổ chức các tiến trình thành một chuỗi nối với nhau qua Unix pipe. Mỗi tiến trình phụ trách một số nguyên tố, loại các bội số của nó và chuyển những số còn lại đến tiến trình kế tiếp. Cách triển khai này phù hợp với yêu cầu của bài thực hành xv6 Utilities Lab do MIT cung cấp [1].

Ý tưởng sàng nguyên tố bằng chuỗi tiến trình được tài liệu của MIT ghi nhận là bắt nguồn từ Doug McIlroy, người có đóng góp quan trọng đối với cơ chế Unix pipe [1]. Việc kết nối đầu ra của một tiến trình với đầu vào của tiến trình khác cũng phản ánh rất rõ triết lý thiết kế công cụ Unix [2]. Các tiến trình độc lập trao đổi dữ liệu qua các kênh giao tiếp có sự tương đồng với *Communicating Sequential Processes* (CSP) của C. A. R. Hoare [3]. Trên xv6, chương trình vận dụng các lời gọi hệ thống `pipe()`, `fork()`, `read()`, `write()` và `wait()` để tổ chức, giao tiếp và đồng bộ các tiến trình [4, 5].

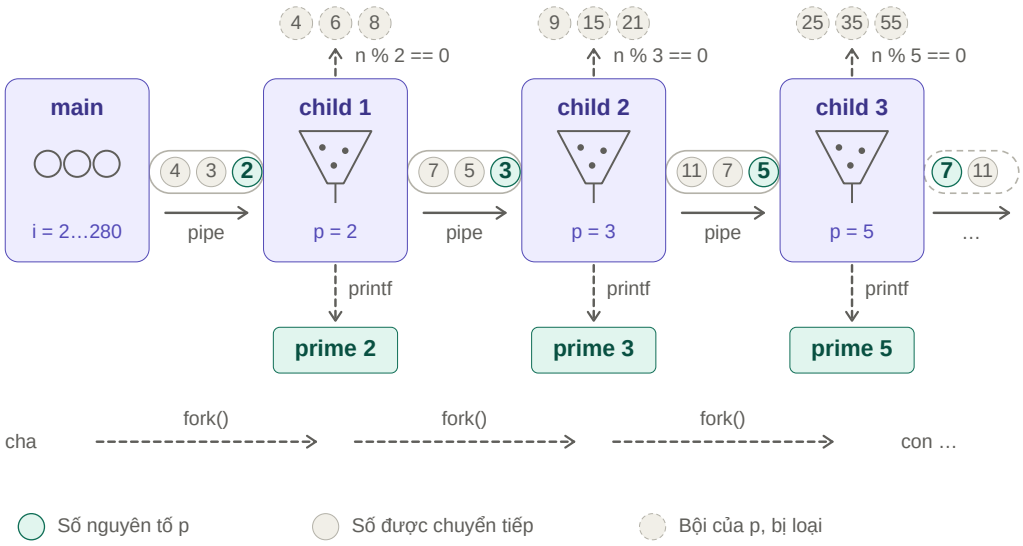


Figure 1: Pipeline sàng nguyên tố đồng thời trong `primes.c`.

Luồng thực thi gồm ba giai đoạn chính:

- **Khởi tạo:** Tiến trình chính tạo `pipe 0`, gọi `fork()` để tạo tầng lọc đầu tiên, sau đó ghi lần lượt các số nguyên từ 2 đến 280 vào đầu ghi của pipe.
- **Lọc số nguyên tố:** Mỗi tầng đọc giá trị đầu tiên p từ pipe bên trái (`left_fd`). Do luồng đầu vào được sắp tăng dần và đã loại bỏ các bội số của các số nguyên tố trước đó, p là một số nguyên tố và được in ra màn hình. Tiến trình tạo một pipe mới (`new_pipe`) và một tiến trình con để đảm nhiệm tầng kế tiếp; các giá trị n thỏa $n \bmod p \neq 0$ được chuyển sang pipe mới. Quá trình này tiếp diễn dọc theo chuỗi pipeline.
- **Kết thúc:** Khi mọi đầu ghi tương ứng đã được đóng và dữ liệu trong pipe đã được đọc hết, `read()` trả về 0 (EOF); giá trị âm biểu thị lỗi đọc, không phải EOF. Từng tiến trình đóng các file descriptor còn sử dụng, gọi `wait(0)` để thu hồi tiến trình con nếu có, rồi thoát. Tiến trình chính chỉ kết thúc sau khi toàn bộ pipeline đã được thu hồi [1, 6].

1.2 Khó khăn và thử thách

Trong quá trình triển khai trên xv6, nhóm cần xử lý ba vấn đề liên quan đến đồng bộ và giới hạn tài nguyên:

1. **Rủi ro về thứ tự đọc, ghi và chờ tiến trình:** Pipe có bộ đệm hữu hạn; khi bộ đệm đầy, `write()` có thể bị chặn để chờ tiến trình khác đọc dữ liệu. Trạng thái chờ này không tự bản thân nó là deadlock mà là rủi ro có thể xuất hiện nếu các tiến trình chờ lẫn nhau, chẳng hạn tiến trình cha gọi `wait()` khi vẫn giữ đầu ghi mà tiến trình con đang đợi EOF [6, 4].

   Giải pháp: Nhóm tạo tiến trình đọc bằng `fork()` trước khi truyền dãy số để dữ liệu được tiêu thụ theo cơ chế streaming. Quan trọng hơn, mỗi tiến trình đóng các đầu pipe không sử dụng và chỉ gọi `wait()` sau khi đã hoàn tất ghi, đóng đầu ghi tương ứng.

2. **Quản lý file descriptor và điều kiện EOF:** Trong xv6, `read()` trên một pipe rỗng chỉ trả về EOF khi không còn đầu ghi nào mở [4]. Do `fork()` kế thừa các file descriptor của tiến trình cha, việc bỏ sót một đầu ghi có thể khiến tiến trình đọc tiếp tục chờ dù dữ liệu đã được truyền hết. Ngoài ra, `NOFILE = 16` giới hạn số file descriptor được mở đồng thời trong mỗi tiến trình [6, 7].

   Giải pháp: Sau `fork()`, tiến trình con đóng đầu ghi của `new_pipe` và những descriptor không còn cần thiết từ tầng trước. Tiến trình cha đóng đầu đọc của pipe mới, chỉ giữ đầu ghi để chuyển các số hợp lệ; khi kết thúc vòng lọc, tiến trình đóng đầu ghi nhằm truyền

EOF cho tầng kế tiếp. Cách quản lý này vừa bảo đảm điều kiện kết thúc, vừa hạn chế số descriptor tồn tại không cần thiết.

3. **Thu hồi tiến trình và giới hạn tài nguyên:** Mỗi số nguyên tố tạo ra một tầng lọc riêng, vì vậy chương trình có thể hình thành một chuỗi gồm hàng chục tiến trình. Trong bản xv6-riscv được tham chiếu, `NPROC = 64` là giới hạn số tiến trình toàn hệ thống. Nếu các tầng không đóng pipe hoặc không được đồng bộ hợp lý, tiến trình có thể tiếp tục bị chặn và chiếm tài nguyên; đồng thời, tiến trình chính thoát quá sớm có thể khiến dấu nhắc shell xuất hiện trước khi mọi kết quả được in xong [7, 1].

   Giải pháp: Nhóm tổ chức việc kết thúc theo chuỗi: mỗi tầng đóng đầu ghi sau khi chuyển hết dữ liệu, chờ tiến trình con của mình bằng `wait(0)` rồi mới thoát. Nhờ đó, việc thu hồi diễn ra từ tầng cuối về tầng đầu và tiến trình chính chỉ trả quyền điều khiển cho shell sau khi pipeline kết thúc.

2 tree.c

2.1 Tổng quan

Lệnh `tree` hiển thị cấu trúc tệp và thư mục dưới dạng cây, giúp quan sát quan hệ phân cấp giữa các thư mục. Phiên bản của nhóm hỗ trợ cờ `-L` để giới hạn độ sâu duyệt và `-d` để chỉ hiển thị thư mục. Chương trình sử dụng duyệt theo chiều sâu (*Depth-First Search*, DFS), kết hợp với cơ chế đọc `struct dirent` và truy xuất thông tin inode của hệ thống tệp xv6 [4, 8].
Luồng thực thi gồm ba giai đoạn:

- **Phân tích đối số:** Chương trình xác định đường dẫn mục tiêu (mặc định là `"."`) và xử lý các tùy chọn `-d`, `-L` kèm độ sâu. Giá trị độ sâu được kiểm tra về cú pháp và nguy cơ tràn số nguyên trước khi sử dụng.

- **Thu thập và lọc mục con:** Chương trình mở thư mục bằng `open()` với `O_RDONLY`, đọc các bản ghi `struct dirent`, bỏ qua `"."`, `".."` và các entry trống có `inum == 0`. Các mục hợp lệ được đưa vào mảng động `children`; việc lọc theo cờ `-d` phải được tính đến trước khi xác định mục cuối để định dạng nhánh chính xác.

- **Định dạng và duyệt đệ quy:** Sau khi thu thập các mục con, chương trình đóng descriptor của thư mục hiện tại trước khi duyệt sâu. Khi hiển thị, chương trình sử dụng trạng thái

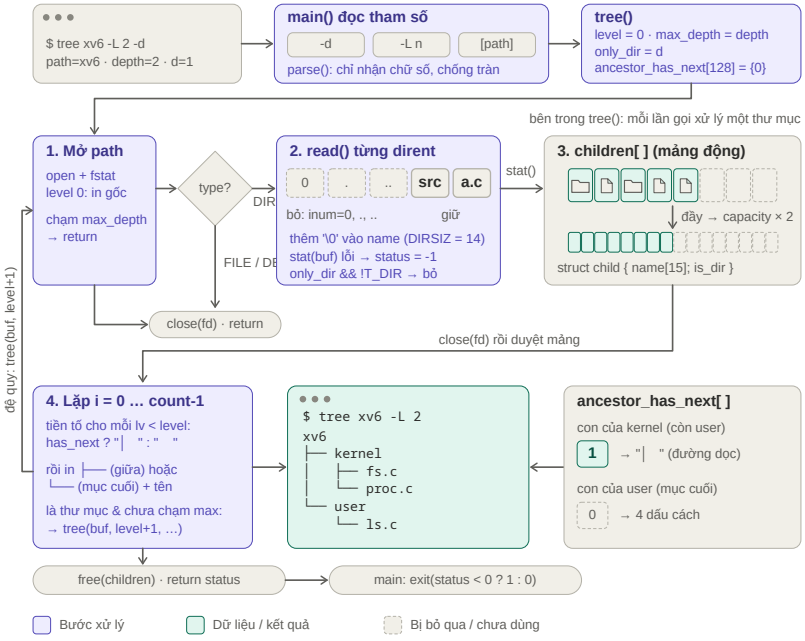


Figure 2: Luồng duyệt DFS và định dạng nhánh cây trong `tree.c`.

`ancestor_has_next` để quyết định vị trí vạch nối dọc, khoảng thụt lề và nhánh cuối; sau đó tiếp tục gọi đệ quy `tree()` cho từng thư mục con hợp lệ.

2.2 Khó khăn và thử thách

Với các giới hạn về descriptor, đường dẫn và cách lưu tên tệp trong xv6, nhóm tập trung xử lý những trường hợp sau:

1. **Giới hạn file descriptor (`NOFILE = 16`):** xv6 giới hạn số descriptor có thể mở đồng thời trong mỗi tiến trình. Nếu giữ descriptor của mọi thư mục cha trong khi đệ quy xuống các thư mục con, số descriptor đang mở có thể tăng theo độ sâu cây, khiến `open()` thất bại khi đạt giới hạn [7].

   Giải pháp: Nhóm đọc các mục con vào mảng động `children`, sau đó gọi `close(fd)` trước khi thực hiện các lời gọi đệ quy. Nhờ vậy, số descriptor mở thêm do thao tác duyệt không tích lũy theo độ sâu cây, giảm nguy cơ vượt giới hạn `NOFILE`.

   Giới hạn còn lại: Mỗi tầng đệ quy vẫn chiếm user stack, gồm bộ đệm `buf[PATHSIZ + 1]` và các biến cục bộ. Trong cấu hình lab hiện tại (`LAB_UTIL`), `USERSTACK = 2` trong

`kernel/param.h`, tương ứng 8192 byte (8 KiB), không tự động mở rộng. Vì vậy, đóng descriptor chỉ giải quyết giới hạn FD; cây quá sâu vẫn có thể làm tràn user stack. Độ sâu an toàn phụ thuộc khung hàm sau biên dịch và phần stack đã sử dụng. Cờ `-L` giới hạn số tầng được duyệt nhưng không mở rộng stack; bỏ cờ này không có nghĩa chương trình hỗ trợ độ sâu tùy ý. Khi không bật `LAB_UTIL`, mã nguồn này dùng `USERSTACK = 1`.

2. **Tên tệp không có ký tự kết thúc chuỗi (`DIRSIZ = 14`):** `struct dirent` lưu tên tệp bằng mảng `name[DIRSIZ]`. Khi tên dài đúng 14 ký tự, mảng không nhất thiết có byte kết thúc `NUL`; vì vậy, xử lý trực tiếp như chuỗi C có thể dẫn đến đọc vượt ranh giới vùng nhớ [8, 4].

   Giải pháp: Nhóm sử dụng bộ đệm `name[DIRSIZ + 1]`, sao chép tối đa `DIRSIZ` byte bằng `memmove()` rồi gán `name[DIRSIZ] = '\0'`. Tên tệp sau đó mới được dùng để hiển thị hoặc ghép đường dẫn.

3. **Kiểm soát kích thước bộ đệm đường dẫn (`PATHSIZ = 128`):** Nhóm định nghĩa `PATHSIZ = 128` và cấp phát bộ đệm `buf[PATHSIZ + 1]` gồm 129 byte. Các lời gọi hệ thống vẫn bị ràng buộc bởi `MAXPATH = 128` (kể cả byte kết thúc chuỗi) trong `kernel/param.h` của xv6-riscv. Khi duyệt nhiều cấp, đường dẫn ghép từ tên thư mục cha và tên mục con có thể vượt kích thước bộ đệm [7].

   Giải pháp: Trước khi ghép đường dẫn, chương trình kiểm tra tổng chiều dài đường dẫn cha, ký tự phân cách, tên con và byte kết thúc chuỗi. Nếu vượt kích thước cho phép, chương trình thông báo lỗi, cập nhật `status = -1`, bỏ qua mục đang xét và tiếp tục các mục khác thay vì kết thúc toàn bộ quá trình duyệt.

4. **Cấp phát bộ nhớ cho danh sách mục con:** Số lượng mục trong mỗi thư mục thay đổi theo dữ liệu thực tế; việc dùng một mảng cố định quá nhỏ có thể làm mất mục, trong khi mảng quá lớn trên stack gây lãng phí bộ nhớ.

   Giải pháp: Mảng `children` được cấp phát trên heap với dung lượng ban đầu bằng 8. Khi số phần tử đạt `capacity`, nhóm cấp phát mảng mới có dung lượng gấp đôi, sao chép các phần tử bằng `memmove()` và giải phóng mảng cũ. Kết quả `malloc()` được kiểm tra trước khi sử dụng; bộ nhớ được thu hồi sau khi hoàn tất duyệt thư mục.

5. **Định dạng nhánh và lưu trạng thái tổ tiên:** Để hiển thị cây đúng cấu trúc, chương trình cần biết ở mỗi cấp tổ tiên còn mục nào phía sau hay không, đồng thời xác định mục hiện tại có phải mục cuối trong danh sách đang hiển thị.

Giải pháp: Nhóm sử dụng mảng `ancestor_has_next[PATHSIZ]` để lưu trạng thái từng cấp. Trước khi gọi đệ quy xuống cấp `level + 1`, chương trình gán `ancestor_has_next[level] = !is_last`. Khi in một dòng, các cấp tổ tiên còn mục tiếp theo được thể hiện bằng vạch dọc, còn các cấp đã kết thúc dùng khoảng trắng. Nhánh hiện tại được chọn theo `is_last`, tính trên danh sách đã áp dụng bộ lọc hiển thị (nếu có).

6. **Kiểm tra đối số và ngăn tràn số nguyên:** Tham số theo sau `-L` có thể là chuỗi rỗng, chứa ký tự không phải chữ số hoặc biểu diễn giá trị vượt giới hạn số nguyên có dấu 32-bit.

   Giải pháp: Hàm `parse(char *s)` kiểm tra từng ký tự và lấy chữ số `digit`; trước khi cập nhật `depth = depth * 10 + digit`, chương trình kiểm tra điều kiện `depth > (INT_MAX - digit) / 10` để phát hiện tràn số. Các trường hợp thiếu giá trị của `-L` hoặc truyền đối số không hợp lệ được xử lý bằng thông báo hướng dẫn sử dụng và mã thoát lỗi.

7. **Xử lý lỗi khi truy xuất hệ thống tệp:** Các thao tác `open()`, `stat()` và `read()` có thể thất bại. Với `read()`, ngoài giá trị âm biểu thị lỗi, chương trình cũng cần phân biệt trường hợp số byte đọc được không đủ kích thước một `struct dirent`; việc duyệt thư mục trong xv6 dựa trên các bản ghi có kích thước cố định [4, 8].

   Giải pháp: Nhóm ghi nhận lỗi bằng biến `status = -1` và tiếp tục xử lý các mục còn lại khi có thể. Nếu `stat()` của một mục thất bại, mục đó được bỏ qua; nếu thao tác đọc thư mục thất bại, chương trình dừng việc đọc thư mục hiện tại, giải phóng tài nguyên và báo lỗi. Mã thoát cuối cùng được xác định bằng `exit(status < 0 ? 1 : 0)`.

3 diff.c

3.1 Tổng quan

`diff.c` được xây dựng nhằm so sánh hai tệp văn bản theo từng dòng và thông báo những vị trí có nội dung khác nhau. Chương trình tham khảo chức năng của `diff` trên Unix và quy ước mã thoát được quy định trong đặc tả POSIX [9]. Tuy nhiên, thay vì tìm một tập phép thêm, xóa hoặc thay đổi dòng để biến đổi tệp thứ nhất thành tệp thứ hai như `diff` tiêu chuẩn, phiên bản của nhóm so sánh trực tiếp các dòng ở cùng vị trí. Vì vậy, đầu ra phản ánh sự khác biệt theo chỉ số dòng, không phải một bản mô tả chỉnh sửa tối thiểu như các thuật toán so sánh tệp truyền thống [10].

Trong xv6, thư viện người dùng chỉ cung cấp một tập hàm tương đối hạn chế, không có sẵn các hàm đọc dòng như `getline()` hay `fgets()` [4]. Do đó, nhóm xây dựng hàm `readline()` để đọc dữ liệu từ file descriptor bằng `read()`, đồng thời quản lý bộ đệm động và trạng thái ký tự xuống dòng.

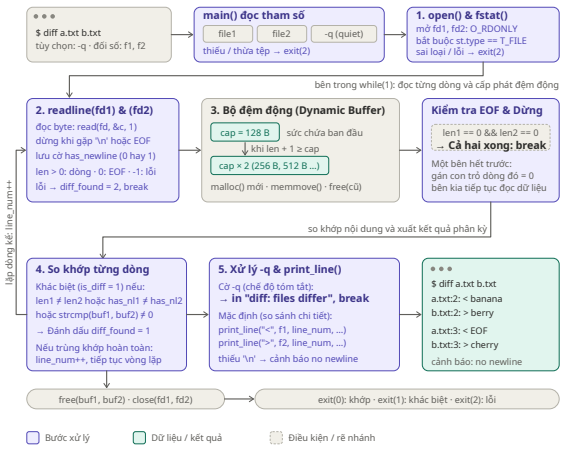


Figure 3: Sơ đồ luồng thực thi và cơ chế so sánh dòng trong `diff.c`

Luồng thực thi gồm các giai đoạn:

- **Mở và kiểm tra tệp:** Phân tích tham số dòng lệnh, mở hai tệp với cờ `O_RDONLY`, sau đó dùng `fstat()` để xác nhận cả hai là tệp thường (`st.type == T_FILE`), tránh xử lý nhầm thư mục hoặc thiết bị.
- **Đọc từng dòng:** Hàm `readline()` sử dụng hai bộ đệm độc lập, mỗi bộ đệm có dung lượng ban đầu 128 byte. Khi một dòng dài hơn sức chứa hiện tại, chương trình cấp phát

vùng nhớ mới với dung lượng gấp đôi, sao chép dữ liệu bằng `memmove()` và giải phóng vùng nhớ cũ. Kết quả của `read()` được lưu riêng: `0` là EOF, giá trị âm là lỗi. Nếu lỗi xảy ra ngay từ byte đầu hoặc giữa dòng, `readline()` trả về `-1`, không coi phần dòng đã đọc là dữ liệu hợp lệ. Hàm chỉ trả về `0` khi gặp EOF mà chưa đọc được byte nào; dòng rỗng có ký tự xuống dòng vẫn có độ dài bằng `1`.

- **So sánh và xuất kết quả:** Với mỗi cặp dòng cùng chỉ số, chương trình đối chiếu chiều dài, cờ `has_newline` và nội dung bằng `strcmp()`. Khi phát hiện khác biệt, hàm `print_line()` hiển thị dòng tương ứng của từng tệp với tiền tố `<` và `>` cùng số dòng để người dùng xác định vị trí sai khác.

3.2 Khó khăn và Thử thách

Trong quá trình triển khai, nhóm tập trung xử lý các trường hợp biên liên quan đến việc đọc dòng và quản lý tài nguyên trên xv6:

1. **Hai tệp có số dòng khác nhau:** Khi một tệp kết thúc trước tệp còn lại, chương trình phải phân biệt được trạng thái hết tệp (EOF) với một dòng rỗng hợp lệ. Nếu chỉ dựa vào độ dài nội dung bằng 0, việc so sánh có thể dừng sớm hoặc bỏ sót những dòng còn lại.
   Giải pháp: Nhóm xử lý riêng trạng thái kết thúc của từng luồng đọc. Hàm `print_line()` cho phép biểu diễn phía đã hết tệp bằng `EOF`, đồng thời tiếp tục đọc và hiển thị những dòng còn lại ở phía kia cho đến khi cả hai luồng đều kết thúc. Cách xử lý này bảo toàn thông tin về các dòng dư, dù không tự động căn chỉnh những đoạn văn bản bị thêm hoặc xóa ở giữa tệp.
2. **Dòng cuối không có ký tự xuống dòng:** Hai dòng có cùng chuỗi ký tự nhưng khác nhau về việc có kết thúc bằng `'\n'` hay không không nên bị xem là hoàn toàn giống nhau. Đây là một trường hợp cần lưu ý khi so sánh tệp văn bản theo dòng [9].
   Giải pháp: Hàm `readline()` lưu cờ `has_newline` cho mỗi dòng và đưa cờ này vào điều kiện so sánh. Nếu dòng cuối không có ký tự xuống dòng, chương trình chủ động xuống dòng khi xuất kết quả và thông báo `no newline at end of file` để phân biệt hai trường hợp.
3. **Chế độ so sánh nhanh và mã thoát:** Khi chỉ cần biết hai tệp có khác nhau hay không, việc in toàn bộ các dòng sai khác gây ra I/O không cần thiết. Tùy chọn `-q` được tham khảo từ chế độ *brief* của GNU Diffutils [11].
   Giải pháp: Khi bật `-q`, chương trình dừng ở khác biệt đầu tiên, đặt `is_diff = 1` và in

thông báo `diff: files differ`. Trước khi thoát, chương trình đóng các file descriptor và giải phóng bộ đệm đã cấp phát. Mã thoát được quy ước là `0` nếu hai tệp giống nhau, `1` nếu có khác biệt và `2` nếu xảy ra lỗi, tương thích với quy ước của `diff` trong POSIX [9]. Cụ thể, thiếu hoặc thừa đường dẫn, mở tệp thất bại, lỗi `fstat()`, đầu vào không phải tệp thường, lỗi cấp phát và lỗi đọc đều dẫn đến mã `2`. Trong vòng so sánh, lỗi đặt `diff_found = 2` rồi dừng, kể cả khi trước đó đã phát hiện khác biệt; các bộ đệm và descriptor được thu hồi trước khi thoát. Với `-q`, chương trình dừng ngay ở khác biệt đầu tiên nên không kiểm tra các lỗi có thể xuất hiện ở phần chưa đọc.

4 du.c

4.1 Tổng quan

`du.c` (*Disk Usage*) tham khảo tiện ích `du` trên các hệ điều hành họ Unix [12], sử dụng thuật toán duyệt đệ quy hậu thứ tự (*post-order bottom-up traversal*) để cộng dồn kích thước tệp trong cây thư mục.

Quy ước tính dung lượng: Phiên bản của nhóm tính theo byte, cộng giá trị `st.size` của các mục `T_FILE` và `T_DEVICE` trong cây con. Chương trình bỏ qua kích thước `st.size` của bản thân mọi thư mục, kể cả thư mục gốc; do đó thư mục rỗng có tổng bằng `0`. Đây là tổng kích thước logic của các tệp, khác với cách thống kê dung lượng khối đĩa của `du` POSIX/GNU, vốn tính cả dung lượng của bản thân thư mục [12]. Quy ước này được áp dụng nhất quán cho đầu ra mặc định và các cờ `-a`, `-s`.

Trong hệ thống tệp xv6 [4], một thư mục được tổ chức như một tập tin đặc biệt chứa danh sách các bản ghi `struct dirent` (16 bytes gồm 2 bytes chỉ mục `inum` và 14 bytes tên `name`). Kích thước byte thực tế `st.size` của từng inode được truy xuất chính xác thông qua lời gọi hệ thống `stat()`.

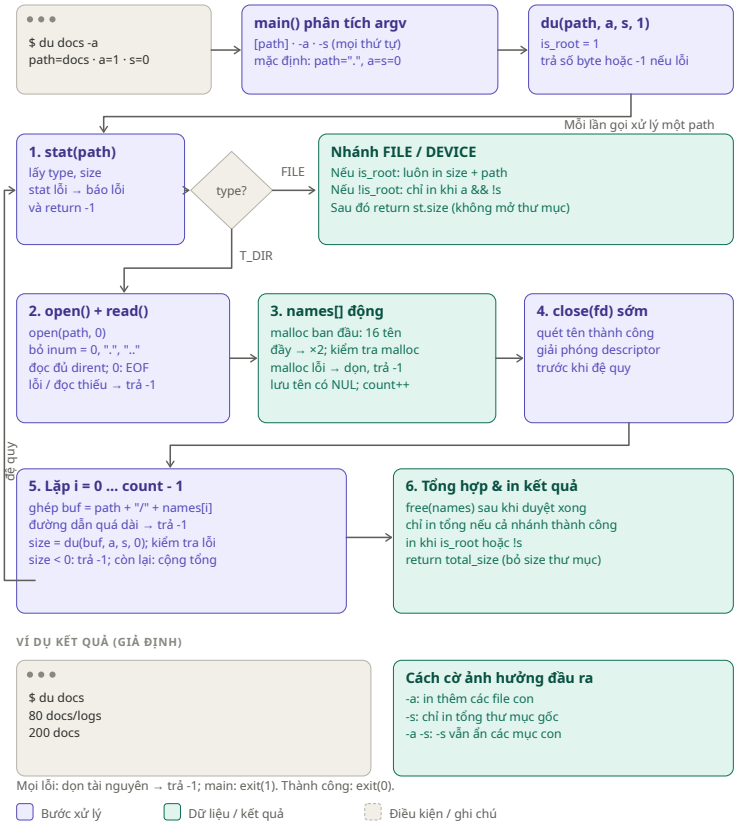


Figure 4: Sơ đồ luồng thực thi duyệt đệ quy Bottom-up và giải phóng FD sớm trong `du.c`

Luồng thực thi diễn ra tuần tự qua các giai đoạn:

- **Khởi tạo và phân tích đối số:** Tiếp nhận tối đa một đường dẫn mục tiêu (mặc định là `"."`), phân tích cờ tóm tắt `-s` (chỉ in kết quả đường dẫn gốc) và cờ chi tiết `-a` (in thêm kích thước tệp con). Khi kết hợp hai cờ, `-s` ẩn mọi kết quả mục con.
- **Điều kiện dừng (Base Case):** Sử dụng `stat(path, &st)`. Nếu đường dẫn trỏ tới một tệp tin thường (`T_FILE`) hoặc tệp thiết bị (`T_DEVICE`), kích thước `st.size` được trả về ngay lập tức; đồng thời in thông tin ra màn hình nếu đường dẫn là gốc được chỉ định hoặc có cờ `-a` (`is_root || (a && !s)`).
- **Duyệt con và cộng dồn (Bottom-up):** Khi gặp thư mục (`T_DIR`), chương trình quét nội dung thư mục, lưu danh sách các tên tệp con hợp lệ, rồi gọi đệ quy `du(buf, a, s, 0)`. Chỉ kết quả không âm mới được cộng vào `total_size`. Sau khi mọi mục con được xử lý thành công, thư mục in tổng nếu là gốc hoặc không bật `-s`, rồi trả tổng về cho cha. Nếu

một mục lỗi, nhánh hiện tại dừng và trả `-1`.

4.2 Khó khăn và Thử thách

Dựa trên kiến trúc của hệ điều hành xv6 và các ca biên trong quản lý tệp tin [4], nhóm nhận thấy các khó khăn như sau:

1. **Giới hạn File Descriptor (`NOFILE = 16`):** Trong kernel xv6, mỗi tiến trình bị giới hạn mở tối đa 16 file descriptors đồng thời (`NOFILE = 16` trong `kernel/param.h`) [4]. Nếu duy trì `fd` của thư mục đang mở trong suốt các lời gọi đệ quy sâu, số FD sẽ tích lũy theo độ sâu, khiến `open()` thất bại khi đạt giới hạn.
   Giải pháp: Đọc toàn bộ tên các mục con vào mảng động `names` trên bộ nhớ heap, sau đó thực hiện lệnh `close(fd)` trước khi bắt đầu bất kỳ lời gọi đệ quy con nào. Nhờ đó, số FD mở thêm do thao tác duyệt duy trì ở mức $O(1)$ (tối đa một FD thư mục trong lúc đọc), không tích lũy theo độ sâu cây.
   Giới hạn còn lại: Mỗi lời gọi `du()` vẫn sử dụng user stack, bao gồm bộ đệm `buf[MAXPATH]` kích thước 512 byte và các biến cục bộ. Trong cấu hình lab hiện tại (`LAB_UTIL`), `USERSTACK = 2` trong `kernel/param.h`, tương ứng 8192 byte (8 KiB); stack này không tự động mở rộng. Vì vậy, chương trình chưa xử lý được cây có độ sâu tùy ý: đệ quy quá sâu vẫn có thể gây tràn user stack. Ngưỡng cụ thể phụ thuộc kích thước khung hàm sau biên dịch và phần stack đã dùng, không chỉ số FD. Khi không bật `LAB_UTIL`, cấu hình trong mã nguồn này dùng `USERSTACK = 1`.
2. **Phòng chống lặp vô hạn và lọc Inode rác:** Mọi thư mục Unix đều chứa hai thực thể liên kết cứng đặc thù là `"."` (thư mục hiện hành) và `".."` (thư mục cha). Ngoài ra, các entry đã xóa trong xv6 vẫn tồn tại trong bảng chỉ mục với `inum == 0`.
   Giải pháp: Bỏ qua các entry có `de.inum == 0` và lọc bỏ `"."` cùng `".."` để không đệ quy quay lại chính thư mục hiện tại hoặc thư mục cha. Điều này ngăn vòng lặp do hai entry đặc biệt gây ra và tránh cộng lặp dung lượng; stack chịu ảnh hưởng khi đệ quy là user stack, vì `du` chạy ở không gian người dùng. Một cây hợp lệ nhưng quá sâu vẫn có thể làm tràn user stack như giới hạn nêu trên.
3. **Quản lý bộ nhớ mảng động và kiểm soát đường dẫn:** Số lượng phần tử trong một thư mục là biến thiên và không thể biết trước, trong khi việc ghép nối chuỗi đường dẫn đệ quy tiềm ẩn nguy cơ tràn mảng đệm.
   Giải pháp: Khai báo `char (*names)[DIRSIZ + 1]`, tức con trỏ tới mảng ký tự có độ dài

cố định. Vùng heap lưu liên tiếp các chuỗi, mỗi phần tử chiếm `DIRSIZ + 1 = 15` byte (gồm byte kết thúc chuỗi), không phải một mảng các con trỏ `char *`. Sức chứa ban đầu là 16 phần tử và tăng gấp đôi (`capacity * 2`) qua `malloc()` và `memmove()` khi danh sách đầy. Cả lần cấp phát đầu và lần mở rộng đều được kiểm tra trước khi sử dụng; nếu thất bại, chương trình báo lỗi, đóng FD, giải phóng mảng cũ nếu có và trả `-1`, không sao chép vào con trỏ rỗng. Trước khi ghép đường dẫn con, chương trình kiểm tra tổng chiều dài đường dẫn cha, dấu phân cách, tên con và byte kết thúc chuỗi không vượt `sizeof(buf)`; mục vượt bộ đệm làm nhánh hiện tại báo lỗi và trả `-1`. Vùng nhớ danh sách được giải phóng sau khi duyệt xong thư mục. Bộ đệm cục bộ 512 byte không mở rộng giới hạn đường dẫn của syscall xv6: `MAXPATH = 128` trong `kernel/param.h` vẫn áp dụng.

4. **Phân biệt tổng hợp lệ với kết quả chưa đầy đủ:** Nếu bỏ qua lỗi đọc hoặc xem mục không truy cập được là có kích thước bằng `0`, chương trình có thể in tổng nhỏ hơn thực tế nhưng vẫn báo thành công.

   Giải pháp: Lưu kết quả `read()` khi đọc thư mục. Chỉ `0` là EOF hợp lệ; giá trị âm hoặc bản ghi ngắn hơn một `struct dirent` đều được báo lỗi. Lỗi `stat()`, `open()`, cấp phát, đọc thư mục hoặc đường dẫn vượt giới hạn được truyền về bằng `-1`. Các tầng cha giải phóng danh sách tên và trả lỗi tiếp, không in tổng của nhánh bị ảnh hưởng. Những dòng của tệp hoặc nhánh đã hoàn tất trước đó vẫn có thể xuất hiện; chương trình không thu hồi đầu ra đã in. Hàm `main()` dùng mã thoát `0` khi tính thành công và `1` khi có lỗi cú pháp hoặc lỗi trong quá trình duyệt.

5 pingpong.c

5.1 Tổng quan

`pingpong.c` minh họa cơ chế giao tiếp liên tiến trình (*Inter-Process Communication* – IPC) và đồng bộ giữa tiến trình cha và tiến trình con trong xv6 [4, 1]. Chương trình thực hiện một lượt trao đổi dữ liệu (*ping-pong*): cha gửi một byte cho con, con nhận rồi gửi lại một byte phản hồi cho cha.

Trong xv6, mỗi pipe là một kênh truyền dữ liệu một chiều dưới dạng luồng byte [4, 6]. Vì vậy, chương trình sử dụng hai pipe độc lập để truyền dữ liệu theo hai hướng: `parent_to_child` (cha ghi, con đọc) và `child_to_parent` (con ghi, cha đọc).

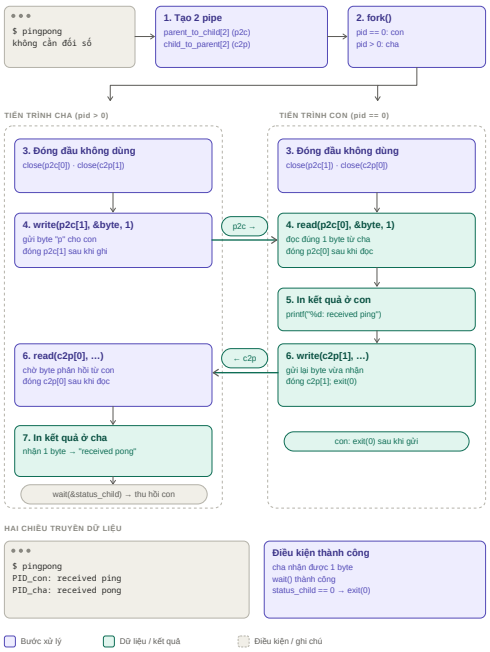


Figure 5: Sơ đồ giao tiếp IPC qua hai pipe và đồng bộ cha – con trong `pingpong.c`

Luồng thực thi gồm các giai đoạn:

- **Khởi tạo kênh truyền:** Tạo hai pipe, sau đó gọi `fork()` để sinh tiến trình con. Hai tiến trình đóng những đầu pipe không sử dụng nhằm tránh giữ file descriptor (FD) không cần thiết (Bảng 1).
- **Trao đổi dữ liệu:** Cha gửi byte `'p'` qua pipe `parent_to_child[1]`. Con đọc dữ liệu từ `parent_to_child[0]`, in dòng `<pid>: received ping`, rồi gửi lại một byte qua

`child_to_parent[1]`. Cha nhận phản hồi từ pipe `child_to_parent[0]` và in `<pid>: received pong`.

- **Thu hồi tài nguyên:** Hai tiến trình đóng các đầu pipe còn sử dụng sau khi hoàn tất I/O. Cha gọi `wait(&status_child)` để chờ và thu hồi tiến trình con, đồng thời kiểm tra kết quả trước khi kết thúc.

Pipe	Cha giữ	Con giữ
`parent_to_child`	Đầu ghi `[1]`	Đầu đọc `[0]`
`child_to_parent`	Đầu đọc `[0]`	Đầu ghi `[1]`

Table 1: Các đầu pipe mỗi tiến trình cần giữ sau `fork()`

5.2 Khó khăn và Thử thách

Trong quá trình hiện thực cơ chế IPC của xv6, nhóm tập trung xử lý các vấn đề về file descriptor, thứ tự trao đổi dữ liệu và quản lý vòng đời tiến trình [4, 5].

1. **Đóng đầu ghi pipe và kiểm soát điều kiện EOF:** Khi pipe rỗng nhưng vẫn còn đầu ghi được mở, `read()` sẽ chờ dữ liệu mới; lệnh chỉ trả về 0 (EOF) khi toàn bộ các đầu ghi đã đóng [4, 6]. Tuy nhiên, với thao tác đọc đúng một byte của chương trình, việc giữ đầu ghi thừa không tự động gây deadlock nếu byte dữ liệu đã đến. Nguy cơ chờ vô hạn xuất hiện khi chương trình tiếp tục đọc trên pipe rỗng để đợi EOF mà vẫn còn một bản sao đầu ghi được giữ mở.

   Giải pháp: Sau `fork()`, con đóng `parent_to_child[1]` và `child_to_parent[0]`; cha đóng `parent_to_child[0]` và `child_to_parent[1]`. Mỗi tiến trình tiếp tục đóng đầu ghi sau khi gửi xong và đóng đầu đọc sau khi nhận xong.
2. **Đảm bảo thứ tự hiển thị giữa hai tiến trình:** Sau `fork()`, lịch chạy của cha và con không cố định. Nếu cho hai tiến trình in kết quả độc lập, thứ tự có thể thay đổi.

   Giải pháp: Sử dụng cơ chế *blocking I/O* của pipe để đồng bộ: con chỉ in `received ping` sau khi nhận byte từ cha, rồi mới gửi phản hồi; cha chỉ in `received pong` sau khi đọc được phản hồi từ con. Vì vậy, thứ tự hai thông báo được xác lập thông qua chính luồng trao đổi dữ liệu, không cần cơ chế khóa bổ sung [4, 1].
3. **Xử lý lỗi khởi tạo và thu hồi tiến trình:** `pipe()` và `fork()` có thể thất bại khi tài nguyên hệ thống không đủ. Trong xv6-riscv, `NPROC = 64` giới hạn số tiến trình, còn `NOFILE = 16` giới hạn số FD mở trên mỗi tiến trình [7]. Tiến trình con đã thoát sẽ ở trạng

thái zombie cho đến khi được thu hồi. Nếu cha thoát trước, xv6 chuyển tiến trình con còn lại cho `init`; đồng thời, `exit()` tự đóng các FD đang mở [4, 13].

Giải pháp: Kiểm tra kết quả mỗi lời gọi `pipe()` và `fork()`. Nếu khởi tạo thất bại, chương trình đóng các FD đã tạo rồi thoát với mã lỗi. Trong luồng thành công, cha gọi `wait()` để thu hồi con, tránh để zombie tồn tại trong thời gian cha còn hoạt động.

6 cp.c

6.1 Tổng quan

`cp.c` hiện thực chức năng sao chép nội dung của một tệp thông thường sang một đường dẫn đích trong xv6, tham khảo tiện ích `cp` của POSIX [14]. Phiên bản này hỗ trợ cú pháp cơ bản `cp src dst`, không nhằm triển khai toàn bộ tùy chọn và trường hợp sử dụng của POSIX `cp`.

Trong xv6, thông tin tệp được quản lý bằng inode; cặp giá trị `(dev, ino)` giúp xác định hai đường dẫn có đang tham chiếu đến cùng một inode hay không [4]. Chương trình đọc dữ liệu từ tệp nguồn được mở bằng `O_RDONLY` và ghi sang tệp đích bằng tổ hợp cờ `O_WRONLY` | `O_CREATE` | `O_TRUNC`, thông qua bộ đệm có kích thước cố định.

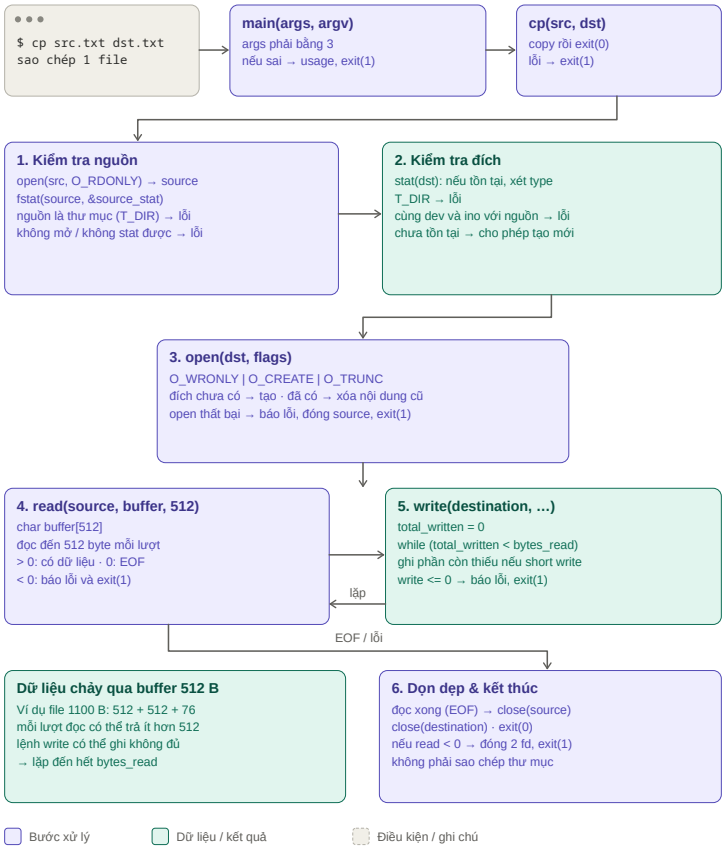


Figure 6: Sơ đồ kiểm tra inode và sao chép dữ liệu theo khối trong `cp.c`

Luồng thực thi gồm các giai đoạn:

- **Kiểm tra đối số và tệp:** Yêu cầu đúng hai đối số đường dẫn (`argc == 3`), mở tệp nguồn bằng `O_RDONLY` và dùng `fstat()` để từ chối thư mục. Nếu đích đã tồn tại, dùng `stat()` kiểm tra kiểu tệp và so sánh cặp `dev`, `ino` nhằm phát hiện trường hợp trùng tệp.
- **Mở đích và chuẩn bị bộ đệm:** Sau khi kiểm tra, mở tệp đích với tổ hợp các cờ `O_WRONLY` | `O_CREATE` | `O_TRUNC`. Sử dụng bộ đệm `buffer[512]` để truyền dữ liệu theo từng khối.
- **Sao chép dữ liệu:** Lặp lại việc `read()` tối đa 512 byte từ nguồn, rồi ghi dữ liệu đã đọc sang đích. Vòng lặp ghi kiểm tra số byte được ghi và tiếp tục nếu còn dữ liệu chưa ghi hết. Khi `read()` trả về 0 (EOF), chương trình đóng các FD và kết thúc thành công; nếu I/O trả về lỗi, chương trình chuyển sang nhánh xử lý lỗi.

6.2 Khó khăn và Thử thách

Trong quá trình xây dựng thao tác sao chép tệp trên xv6, nhóm chú trọng việc tránh ghi đè nhầm tệp nguồn, kiểm tra kết quả I/O và dọn dẹp tài nguyên [4].

1. **Ngăn việc ghi đè vào chính tệp nguồn:** Nếu nguồn và đích trỏ đến cùng một inode, thao tác mở đích với `O_TRUNC` sẽ cắt kích thước của tệp thường về 0, khiến dữ liệu nguồn bị mất trước khi sao chép [15].
   Giải pháp: Trước khi mở đích, chương trình dùng `stat()` để đối chiếu: `source_stat.dev` == `destination_stat.dev` cùng với `source_stat.ino` == `destination_stat.ino`. Nếu cả hai điều kiện trên đều thỏa mãn, chương trình thông báo lỗi và đóng FD nguồn mà không thực hiện `O_TRUNC`.
2. **Kiểm tra kết quả ghi dữ liệu:** Trong giao diện I/O kiểu Unix, mã trả về của `write()` cần được kiểm tra thay vì giả định mọi lần ghi đều hoàn tất. Riêng trên xv6, đường ghi của tệp thông thường xử lý nhiều khối trong kernel và thường trả về đủ số byte yêu cầu hoặc mã lỗi [16].
   Giải pháp: Dùng biến `total_written` để theo dõi lượng dữ liệu đã ghi. Nếu `write()` trả về số byte dương nhỏ hơn số byte cần ghi, chương trình tiếp tục ghi tiếp từ vị trí `buffer` + `total_written`; nếu trả về giá trị không dương, chương trình xử lý như lỗi ghi.
3. **Từ chối sao chép thư mục và dọn dẹp FD:** Thư mục trong xv6 chứa dữ liệu cấu trúc hệ thống tệp. Việc sao chép thô các byte này sang tệp thường không tương đương với sao chép cây thư mục [4].
   Giải pháp: Dùng `fstat()` hoặc `stat()` để từ chối nguồn hoặc đích có kiểu `T_DIR`. Khi `open()`, `read()` hoặc `write()` thất bại, chương trình đóng các FD đã mở và trả về mã lỗi, thay vì tiếp tục với dữ liệu chưa được sao chép đầy đủ.

References

[1] MIT CSAIL PDOS Group, "Lab: xv6 and Unix utilities," MIT 6.1810: Operating System Engineering, https://pdos.csail.mit.edu/6.1810/2024/labs/util.html, 2024, section: primes; accessed 2026-10-16.

[2] M. D. McIlroy, E. N. Pinson, and B. A. Tague, "UNIX time-sharing system: Foreword," *The Bell System Technical Journal*, vol. 57, no. 6, pp. 1899–1904, 1978, full text: https://www.nokia.com/bell-labs/publications-and-media/publications/unix-time-sharing-system-forward/.

[3] C. A. R. Hoare, "Communicating sequential processes," *Communications of the ACM*, vol. 21, no. 8, pp. 666–677, 1978, https://doi.org/10.1145/359576.359585.

[4] R. Cox, F. Kaashoek, and R. Morris, *xv6: A Simple, Unix-like Teaching Operating System*, 2024, rISC-V edition, revision 4 (August 31, 2024). https://pdos.csail.mit.edu/6.828/2024/xv6/book-riscv-rev4.pdf.

[5] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*. Arpaci-Dusseau Books, 2018, version 1.00 (August 2018). Free PDF: https://pages.cs.wisc.edu/~remzi/OSTEP/.

[6] MIT PDOS, "xv6-riscv source code: kernel/pipe.c," GitHub repository, https://github.com/mit-pdos/xv6-riscv/blob/riscv/kernel/pipe.c, implements pipe read, write, and close; branch riscv; accessed 2026-10-16.

[7] ——, "xv6-riscv source code: kernel/param.h," GitHub repository, https://github.com/mit-pdos/xv6-riscv/blob/riscv/kernel/param.h, defines NPROC, NOFILE, and MAXPATH; branch riscv; accessed 2026-10-16.

[8] ——, "xv6-riscv source code: kernel/fs.h," GitHub repository, https://github.com/mit-pdos/xv6-riscv/blob/riscv/kernel/fs.h, defines DIRSIZ and struct dirent; branch riscv; accessed 2026-10-16.

[9] IEEE and The Open Group, *diff: Compare Two Files*, issue 7, 2018 edition ed., The Open Group Base Specifications, 2018, iEEE Std 1003.1-2017. https://pubs.opengroup.org/onlinepubs/9699919799/utilities/diff.html.

[10] J. W. Hunt and M. D. McIlroy, "An algorithm for differential file comparison," Bell Laboratories, Computing Science Technical Report 41, Jul. 1976, author-hosted copy: https://www.cs.dartmouth.edu/~doug/diff.pdf.

[11] Free Software Foundation, *GNU Diffutils Manual: Summarizing Which Files Differ*, GNU Project, section 1.6, option -q / –brief; accessed 2026-10-16. [Online]. Available: https://www.gnu.org/software/diffutils/manual/html_node/Brief.html

[12] IEEE and The Open Group, *du: Estimate File Space Usage*, issue 7, 2018 edition ed., The Open Group Base Specifications, 2018, iEEE Std 1003.1-2017. https://pubs.opengroup.org/onlinepubs/9699919799/utilities/du.html.

[13] MIT PDOS, "xv6-riscv source code: kernel/proc.c," GitHub repository, https://github.com/mit-pdos/xv6-riscv/blob/riscv/kernel/proc.c, process exit, reparenting, and wait; branch riscv; accessed 2026-10-16.

[14] IEEE and The Open Group, *cp: Copy Files*, issue 7, 2018 edition ed., The Open Group Base Specifications, 2018, iEEE Std 1003.1-2017. https://pubs.opengroup.org/onlinepubs/9699919799/utilities/cp.html.

[15] MIT PDOS, "xv6-riscv source code: kernel/sysfile.c," GitHub repository, https://github.com/mit-pdos/xv6-riscv/blob/riscv/kernel/sysfile.c, open system call and O_TRUNC behavior; branch riscv; accessed 2026-10-16.

[16] ——, "xv6-riscv source code: kernel/file.c," GitHub repository, https://github.com/mit-pdos/xv6-riscv/blob/riscv/kernel/file.c, file descriptor layer and filewrite; branch riscv; accessed 2026-10-16.